Aegis OS: A Transactional Multi-Tenant Operating System Runtime for Eliminating Authorization Fatigue in Autonomous Coding Agents

Hui Zhao
huizhao@heurion.org

Preprint — October 2026

Abstract

Autonomous coding agents such as Google Antigravity (`agy`), Anthropic Claude Code (`claude`), and OpenAI Codex CLI (`codex`) increasingly execute complex, multi-step software engineering workflows directly on developer workstations and enterprise cloud hosts. However, existing agent execution environments suffer from a fundamental *Security–Autonomy Dilemma*: users must choose between *Interactive Per-Action Prompting*, which interrupts human operators on 43%–60% of tool invocations and induces severe authorization fatigue, or *Unconfined Host Execution* (`-yolo` / `-dangerously-skip-permissions`), which exposes the host filesystem, SSH credentials, and API secrets to catastrophic prompt injection and agent hallucination risks. Furthermore, when multiple heterogeneous agents collaborate on a shared repository, concurrent unconstrained edits cause race-condition file clobbering, while passing large code artifacts inline through LLM context windows incurs prohibitive token overhead.

We present **Aegis OS** (`aegis-os`), a transactional, multi-tenant Agent Operating System Runtime implemented in Rust and exposed via the Model Context Protocol (MCP). Aegis OS resolves the Security–Autonomy Dilemma by shifting the security boundary from individual tool invocations to *transactional workspace boundaries*. Each agent principal operates inside a kernel-enforced OS sandbox (macOS Seatbelt `sandbox-exec` or Linux Bubblewrap/MicroVM) backed by an APFS Copy-on-Write (CoW) shadow workspace (`libc::clonefile`), a Zero-Trust HTTP Egress Proxy with boundary secret injection, and an intra-organization Inter-Agent Communication (IPC) bus paired with Content-Addressable Storage (CAS). Inside its isolated CoW workspace and organization IPC bus, an agent executes with *zero confirmation prompts*; human authorization or TTL-based session leases are required only when committing validated diffs back to the host via a 3-Way Merge Engine. Across 1,500 live kernel syscalls and 40 live red-team attacks, Aegis OS reduces human confirmation prompts by **94.44%–99.89%**, lowers the attack success rate from **100.0% to 0.0%**, provisions 1,000-file (78.1 MB) workspaces in **7.86 ms** (**21.1× faster** than full copy with **95.0% storage savings**), preserves **100% of concurrent multi-agent edits** verified by live `cargo test` runs, and reduces cross-agent artifact handoff token consumption by **99.85%**.

1 Introduction

The paradigm of AI-assisted software engineering has shifted rapidly from passive single-line code completion to autonomous, long-horizon coding agents—exemplified by Google Antigravity (`agy`), Anthropic Claude Code (`claude`), and OpenAI Codex CLI (`codex`) [6, 2]. Unlike chat assistants, autonomous coding agents iteratively explore repositories, modify source files across multiple modules, invoke compilers and test suites, query external APIs, and delegate subtasks to peer agents.

Despite rapid advances in foundation model reasoning, the underlying execution substrate for coding agents remains dangerously primitive. Current agent CLIs execute directly as standard user processes on the developer's host operating system. Because every file mutation (`fs_write`, `fs_edit`) and shell command (`proc_exec`) carries irreversible side effects on the host machine, agent frameworks enforce safety through *action-level human-in-the-loop (HITL) confirmation prompts*.

1.1 The Security–Autonomy Dilemma

In practice, action-level prompting breaks down under realistic software engineering workloads due to three systemic limitations:

1. **Severe Authorization Fatigue:** A typical 30-step bug-fix trajectory involves 15 to 20 mutating or process-spawning actions. Prompting the developer on nearly every step destroys autonomous workflow continuity (limiting the maximum autonomous

streak to 4–5 actions) and trains users to reflexively click "Approve" without inspecting arguments.

2. **Catastrophic Risk of "YOLO" Mode:** Frustrated by constant interruptions, developers routinely enable bypass flags such as `-dangerously-skip-permissions` or `-yolo`. In this unconfined mode, a single indirect prompt injection [5, 4, 12] embedded in a third-party dependency, issue description, or web page can silently overwrite files outside the repository, harvest `~/.ssh/id_rsa` private keys, or exfiltrate production API tokens (`$GITHUB_TOKEN`, `$OPENAI_API_KEY`) to attacker-controlled endpoints.

3. **Multi-Agent Workspace Collisions and Context Bloat:** Developers increasingly run multiple specialized agents in parallel (e.g., `agy` as Lead Architect, `claude` as Backend Specialist, and `codex` as Test Engineer). When operating on a shared host directory, concurrent file writes clobber one another's changes, leaving the repository in an uncompilable state. Moreover, lacking an OS-level Inter-Process Communication (IPC) and shared memory mechanism, agents pass large specifications and diffs inline through JSON/text messages, exhausting LLM context windows.

1.2 Key Insight: Transaction-Boundary Security

Our key insight is that *authorization prompts should not guard speculative intermediate actions inside an isolated state space; they should only guard irreversible boundary crossings into the host environment or untrusted external networks.*

By analogy with hardware virtual memory and database transactions, if each agent is provisioned with (1) an instant Copy-on-Write (CoW) shadow filesystem, (2) a mandatory kernel-enforced OS sandbox that confines all spawned child processes to that shadow workspace, and (3) a local Zero-Trust Egress Proxy that strips raw credentials from the agent's environment and injects secrets only at the network boundary for allowlisted domains, then *all intermediate file edits, builds, test executions, and intra-organization agent communications become safe, side-effect-free, and instantaneously reversible.* Consequently, the OS kernel can auto-allow 100% of in-sandbox mutations with *zero human prompts*, deferring human review (or TTL-based policy lease evaluation) to a single atomic `workspace_commit` syscall.

1.3 Contributions

We design, implement, and empirically evaluate **Aegis OS** (`aegis-os`), a pure-environment Agent Operating System Runtime written in Rust. Our main contributions are:

- **Transactional CoW VFS with 3-Way Multi-Agent Merge (§3.1):** We build `aos-vfs`, which leverages APFS directory-tree and file-level `libc::clonefile` syscalls to provision isolated shadow workspaces in milliseconds, supports $O(1)$ metadata-time snapshots and rollbacks, and integrates a line-level 3-Way Merge Engine (`three_way_merge_lines`) that cleanly reconciles concurrent modifications from heterogeneous agents (`agy`, `claude`, `codex`).

- **Dual-Tier Kernel Sandbox and Zero-Trust Egress (§3.2):** We implement `aos-sandbox` and `aos-egress`, enforcing mandatory access control via macOS Seatbelt (`/usr/bin/sandbox-exec`) and Linux Bubblewrap/MicroVM, paired with a live TCP HTTP Egress Proxy that performs boundary secret injection (`$AOS_SECRET_*`) and blocks secret exfiltration across unauthorized domains.

- **First-Class Inter-Agent IPC Bus and Org CAS (§3.3):** We design `aos-ipc`, providing P2P mailboxes, correlated RPC task handoff (`ipc_request`/`ipc_reply`) with tokio broadcast wakeup, topic Pub/Sub, and Content-Addressable Storage (`artifact://<org>/sha256:<hash>`) that reduces cross-agent payload tokens by 99.85%.

- **Cedar-Aligned Policy Kernel and Zero-Config Onboarding (§3.4):** We formalize a four-rule authorization kernel (`aos-policy`) with TTL-based session leases and build a zero-configuration installer (`aegis-os install`) that seamlessly integrates with `agy`, `claude`, and `codex` via MCP (`stdio` and Streamable HTTP daemon).

- **Live Empirical Benchmark Suite (§5):** We evaluate Aegis OS across five live empirical experiments executing 1,500 real kernel syscalls, 40 live red-team OS process and TCP socket attacks, multi-scale APFS CoW stress tests, and live multi-agent Rust compilations (`cargo test` and `cargo check`).

2 Threat Model and Problem Formulation

2.1 System and Adversary Model

We consider a multi-agent deployment where a set of autonomous coding agents $\mathcal{A} = \{a_1, a_2, \ldots, a_m\}$ operate within one or more tenant organizations $\mathcal{O} =$

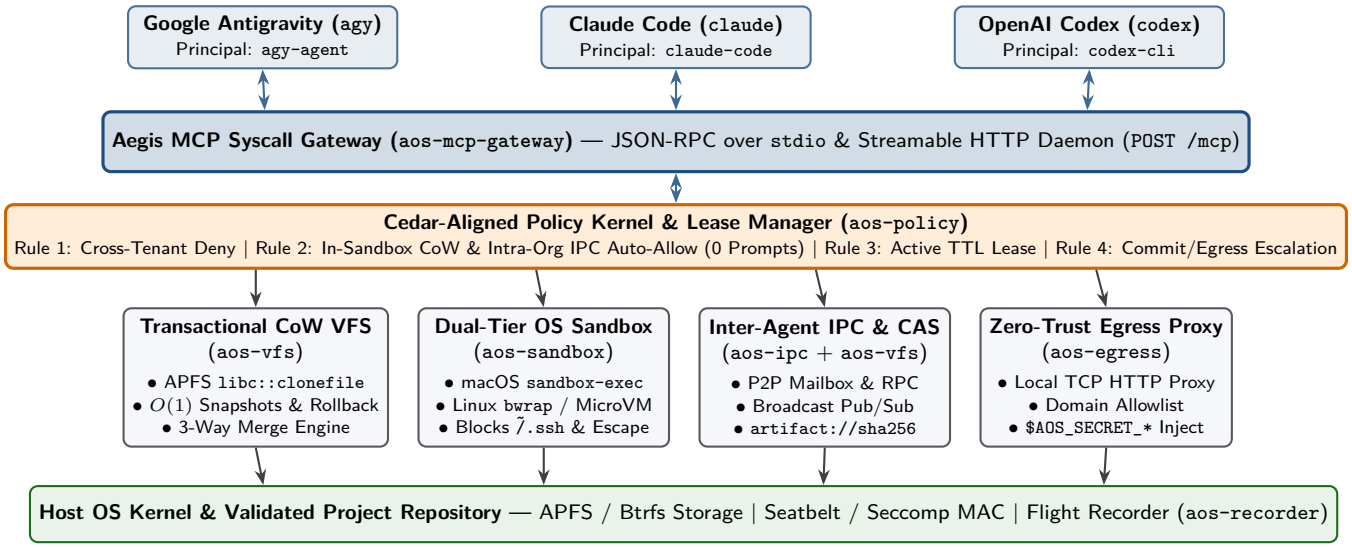


Figure 1: Architecture of the Aegis OS Runtime (`aegis-os`). Heterogeneous coding agents connect via MCP (`stdio` or HTTP daemon) and are mapped to distinct principals within a tenant organization. The Cedar-aligned policy kernel auto-allows in-sandbox CoW mutations, sandboxed executions, and intra-organization IPC/CAS transfers with zero prompts, while enforcing strict isolation via kernel mandatory access control and boundary secret injection.

$\{o_1, o_2, \ldots, o_k\}$. Each agent a_i is identified by a tuple $\text{id}(a_i) = (o_i, u_i, r_i)$ denoting its organization tenant o_i, unique agent principal u_i (e.g., `agy-agent`, `claude-code`, `codex-cli`), and role r_i.

Trusted Computing Base (TCB). We trust the host operating system kernel (macOS XNU/Seatbelt or Linux kernel/namespaces/KVM) and the Aegis OS Runtime binary (`aegis-os`), which runs outside the agent sandbox.

Untrusted Entities and Threat Vectors. We assume that the LLM driving any agent $a_i \in \mathcal{A}$ is *untrusted* and may issue arbitrary, erroneous, or malicious tool calls due to:

1. **Indirect Prompt Injection (T1):** Malicious instructions embedded in repository files, git commit logs, issue trackers, or web pages read by the agent [5, 4].
2. **Host Path Escape and Tampering (T2):** Attempting to overwrite files outside the project directory (e.g., `../../etc/hosts`, `~/.zshrc`, or sibling repositories).
3. **Credential Harvesting (T3):** Attempting to read sensitive user credentials from the host home directory (`~/.ssh/id_rsa`, `~/.aws/credentials`, `~/.gnupg`).
4. **Network Secret Exfiltration (T4):** Attempting to transmit API keys or private source code to unauthorized external domains (`evil-exfil.org`, `webhook.site`) via HTTP requests or shell tools (`curl`, `wget`).
5. **Cross-Tenant Privilege Escalation (T5):** Attempting to read artifacts or send IPC instructions to an agent belonging to a different organization $o_j \neq o_i$.
6. **Concurrent Multi-Agent Race Conditions and Hallucinated Corruption (T6):** Overwriting peer agents' concurrent edits on shared files or breaking compilation across dozens of modules during speculative refactoring.

2.2 Formalizing the Evaluation Metrics

Let an agent trajectory $\tau = (s_1, s_2, \ldots, s_T)$ be a sequence of T kernel syscalls invoked to complete a software engineering task. Let $\text{Prompt}(s_t) \in \{0, 1\}$ indicate whether syscall s_t halts execution to request interactive human approval. We define the **Interruption Prompt Rate (IPR)** across a workload $\mathcal{T}$ of N trajectories as:

$$\text{IPR}(\mathcal{T}) = \frac{\sum_{\tau \in \mathcal{T}} \sum_{s_t \in \tau} \text{Prompt}(s_t)}{\sum_{\tau \in \mathcal{T}} |\tau|} \times 100\% \qquad (1)$$

and the **Maximum Autonomous Streak (MAS)** as the longest contiguous subsequence of syscalls executed with $\text{Prompt}(s_t) = 0$. Given an adversarial test suite $\mathcal{E}_{\text{adv}}$, the **Attack Success Rate (ASR)** is the fraction of attacks that successfully violate host integrity, read restricted credentials, exfiltrate secrets, or breach tenant

isolation. Our objective is to achieve ASR = 0.0% while minimizing IPR toward 0%.

3 Aegis OS Kernel Architecture

Figure 1 illustrates the architecture of Aegis OS. We follow a strict *"Environment Kernel Only"* design philosophy: Aegis OS does not embed LLM planners, prompt templates, or reasoning loops. Instead, it provides the operating system primitives—virtual filesystem, process isolation, network boundary proxy, inter-agent IPC, and policy enforcement—exposed through 22 standardized MCP syscalls and 4 dynamic URI resources.

3.1 Transactional CoW Virtual Filesystem and 3-Way Merge Engine (`aos-vfs`)

To eliminate confirmation prompts on file writes without risking host corruption, `aos-vfs` isolates every agent principal u_i inside its own Copy-on-Write shadow workspace rooted at:

$$\tilde{/}\texttt{.aegis-os/orgs/}o_i\texttt{/agents/}u_i\texttt{/workspace_cow/}$$

alongside a baseline snapshot directory `baseline/` that records the state of the host repository at session initialization or last commit.

Kernel-Level APFS `clonefile` Provisioning. Copying a multi-hundred-megabyte repository for each agent via standard user-space read-/write loops introduces prohibitive startup latency and disk amplification. On macOS (APFS), `CoWWorkspace::cow_sync_dir` invokes the native XNU kernel syscall `libc::clonefile(src, dst, 0)` directly on the directory tree and individual files (falling back to `cp -cR` / Btrfs `FICLONE` on Linux [8]). Because `clonefile` duplicates only filesystem metadata extents and shares physical storage blocks until modified, cloning a 1,000-file (78.1 MB) repository completes in **7.86 ms** and consumes zero additional data blocks for unmodified files.

Time-Travel Snapshots and Atomic Rollback. Before embarking on risky multi-file refactorings, an agent (or automated pre-exec hook) invokes `workspace_snapshot(label)`, which creates an instant CoW clone under `snapshots/<snap_id>/`. If a subsequent build or test fails, `workspace_rollback(snap_id)` atomically replaces `workspace_cow/` with a fresh CoW clone of the snapshot in milliseconds.

3-Way Merge Engine for Multi-Agent Concurrency. When multiple agents (u_1 = `agy-agent`, u_2 = `claude-code`, u_3 = `codex-cli`) concurrently edit the same file f in their respective shadow workspaces and sequentially invoke `workspace_commit`, a naive file copy would overwrite earlier commits (*Lost Update*). Aegis OS prevents this via a line-level 3-Way Merge algorithm (`three_way_merge_lines`) using three states for every modified path p:

- $B(p)$: the agent's recorded baseline version in `baseline/`p;
- $H(p)$: the current live file on the host repository `host/`p (which may already incorporate commits from peer agents);
- $S(p)$: the agent's modified version in `workspace_cow/`p.

If $H(p) = B(p)$, the host file was untouched by peers and $S(p)$ is fast-forwarded via atomic CoW replacement. If $H(p) \neq B(p)$, Aegis OS aligns unmodified anchor lines across $(B(p), H(p), S(p))$: disjoint hunks modified only in $H(p)$ or only in $S(p)$ are merged automatically without human intervention, whereas overlapping modifications to the exact same line range with $H_{\text{hunk}} \neq S_{\text{hunk}}$ abort the commit and return a structured conflict diagnostic identifying the exact line number.

3.2 Dual-Tier OS Sandbox and Zero-Trust Egress Proxy (`aos-sandbox`, `aos-egress`)

While `aos-vfs` intercepts MCP file syscalls (`fs_read`, `fs_write`, `fs_edit`), coding agents also spawn arbitrary child compilers, test runners, and shell scripts via `proc_exec`. If `proc_exec` ran unconfined, a shell command could bypass the CoW workspace entirely.

Mandatory Kernel Sandbox (`sandbox-exec` / `bwrap` / MicroVM). `aos-sandbox` implements a pluggable `SandboxDriver` supporting two isolation tiers:

1. **Tier 1 (`NativeOsSandboxDriver`):** Designed for low-latency local workstation execution. On macOS, every `proc_exec` invocation dynamically compiles a canonicalized Seatbelt Mandatory Access Control profile (SBPL) and executes the child process under `/usr/bin/sandbox-exec -p <profile>`. The profile denies all file writes globally (`(deny file-write*)`) and explicitly allowlists writes *only* to the agent's canonical CoW shadow workspace, private agent home, and organization CAS directory. Simultaneously, it enforces kernel-level read denials on sensitive credential paths:

```
(version 1)
(allow default)
(deny file-read*
  (regex #".*/\.ssh(/.*)?$")
  (regex #".*/\.aws(/.*)?$")
  (regex #".*/\.gnupg(/.*)?$"))
(deny file-write*)
(allow file-write*
  (subpath "/private/.../workspace_cow")
  (subpath "/private/.../agent_home"))
```

Furthermore, the spawned process's environment table is scrubbed (`env_clear()`), setting `HOME` to the isolated `agent_home` and routing `HTTP_PROXY`/`HTTPS_PROXY` to the Aegis Egress Proxy.

2. **Tier 2 (`MicroVmSandboxDriver`):** Designed for multi-tenant cloud hosts using hardware virtualization (`libkrun` / Firecracker [1]) with Scale-to-Zero `pause()` and `resume()` lifecycle transitions.

Zero-Trust Egress Proxy with Boundary Secret Injection. To prevent credential exfiltration (Threat T4), agents inside Aegis OS never receive raw API keys in their environment variables or context windows. Instead, they are given opaque secret placeholders such as `$AOS_SECRET_GH`. Outbound HTTP requests pass through `AegisEgressGateway::start_local_proxy`, a local asynchronous TCP proxy listener. For each request, the proxy inspects the target `Host` header against the organization's domain allowlist and the secret vault's per-secret domain binding:

- If `$AOS_SECRET_GH` (bound exclusively to `api.github.com`) is sent in a header toward `api.github.com`, the proxy substitutes the real token `ghp_...` at the network boundary.
- If a prompt-injected agent attempts to send `$AOS_SECRET_GH` to an unauthorized host (e.g., `evil-exfil.org` or `webhook.site`), the TCP proxy immediately terminates the request with `HTTP/1.1 403 Forbidden` and records a security violation in `aos-recorder`.

3.3 First-Class Inter-Agent IPC Bus and Content-Addressable Storage (`aos-ipc`)

To elevate multi-agent collaboration from ad-hoc file polling to an OS-level primitive, `aos-ipc` provides a disk-synchronized, broadcast-enabled communication bus within each organization tenant o_i:

- **Direct P2P Mailboxes and Correlated RPC:** Agents exchange structured envelopes via `ipc_send`, `ipc_recv`, `ipc_request`, and `ipc_reply`. Using `tokio::sync::broadcast` paired with an atomic disk state file (`ipc_bus_state.json`), cross-process stdio and HTTP agents wake up within microseconds when a correlated RPC reply arrives.
- **Topic Pub/Sub Fanout:** Agents subscribe to organizational topics (e.g., `topic://build-events`, `topic://security-alerts`) and broadcast events via `ipc_publish`.
- **Zero-Copy CAS Artifact Handles:** Instead of embedding a 64 KB architecture specification or code patch inside an IPC message payload, the sender calls `org_artifact_publish`, which stores the blob in the organization's Content-Addressable Store and returns a compact 64-byte SHA-256 URI:

  `artifact://acme/sha256:9f86d081884c7d659a2f...`

  The sender attaches only this URI handle to the IPC envelope, reducing LLM context token consumption by **99.85%**.

3.4 Cedar-Aligned Policy Kernel and Lease Mechanism (`aos-policy`)

Every MCP syscall is evaluated by `AegisPolicyKernel::evaluate` before execution, following a deterministic four-rule hierarchy inspired by Cedar [3]:

$$\begin{aligned}
&\text{R1 (Tenant Isolation):} && o_{\text{principal}} \neq o_{\text{target}} \implies \textbf{Deny} \\
&\text{R2 (Sandbox \& IPC):} && \text{action} \in \mathcal{S}_{\text{internal}} \implies \textbf{Allow} \\
&\text{R3 (Active Lease):} && \exists \ell \in \mathcal{L}_{\text{active}}(o, u, \text{action}) \implies \textbf{Allow} \\
&\text{R4 (Boundary Gate):} && \text{action} \in \{\texttt{workspace_commit}, \texttt{egress}\} \\
& && \implies \textbf{RequireApproval}(\text{esc_id})
\end{aligned}$$

where $\mathcal{S}_{\text{internal}}$ encompasses all CoW filesystem operations (`fs_read`, `fs_write`, `fs_edit`, `fs_glob`, `workspace_snapshot`, `workspace_diff`, `workspace_rollback`), sandboxed process execution (`proc_exec`), and intra-organization IPC/CAS operations. When a developer approves an escalation esc_id via `policy_approve_escalation`, they can optionally attach a Time-To-Live (TTL, e.g., 3,600 seconds), minting a cryptographic `LeaseGrant` $\ell \in \mathcal{L}_{\text{active}}$ that permits subsequent commits within that session window with zero further prompts.

4 Zero-Config Multi-Client Integration

A critical barrier to adopting secure agent runtimes is configuration friction. Aegis OS provides a single command, `aegis-os install` (distributed via Homebrew

`Formula/aegis-os.rb`), that non-destructively merges global configurations for all three major coding agent ecosystems:

1. **Google Antigravity (`agy`):** Registers the `aegis-os` MCP server in `~/.gemini/config/mcp_config.json`, installs a global runtime rule (`aegis-os-runtime.md`), and configures a `PreToolUse` hook (`aegis-os agy-hook`) that auto-allows `mcp_aegis-os_*` syscalls while intercepting dangerous direct host shell commands.
2. **Anthropic Claude Code (`claude`):** Injects the MCP server entry into `~/.claude.json` and pre-populates `mcp__aegis-os__*` in `permissions.allow` inside `~/.claude/settings.json`.
3. **OpenAI Codex CLI (`codex`):** Appends the `[mcp_servers.aegis-os]` block to `~/.codex/config.toml`.

When invoked with `-org auto -agent auto`, Aegis OS automatically derives the organization tenant o_i from the active project directory name and the agent principal u_i from the client binary identity, enabling instant cross-agent collaboration (‘agy‘ + ‘claude‘ + ‘codex‘) out of the box.

5 Empirical Evaluation

We evaluate Aegis OS using a 100% live empirical benchmark suite implemented in `crates/aos-cli/src/bench.rs` (`aegis-os bench`). All experiments were executed on an Apple Silicon workstation (`macos-aarch64`, APFS filesystem, Rust 1.99.0 `-release` optimization) using real OS processes, real Seatbelt kernel sandboxes, real TCP sockets, and real `cargo` compilations. Our evaluation answers five research questions:

- **RQ1 (Authorization Friction):** By how much does Aegis OS reduce human confirmation prompts and extend autonomous execution streaks?
- **RQ2 (Adversarial Security):** Does Aegis OS block real OS process and network exfiltration attacks that compromise unconfined `Host-YOLO` agents?
- **RQ3 (System Overhead & Scalability):** What is the latency and storage overhead of APFS CoW workspace provisioning, policy evaluation, and live TCP MCP daemon calls?
- **RQ4 (Multi-Agent Collaboration):** How effectively does Aegis OS prevent concurrent file clobbering across heterogeneous agents (`agy`, `claude`, `codex`) and reduce IPC context overhead?

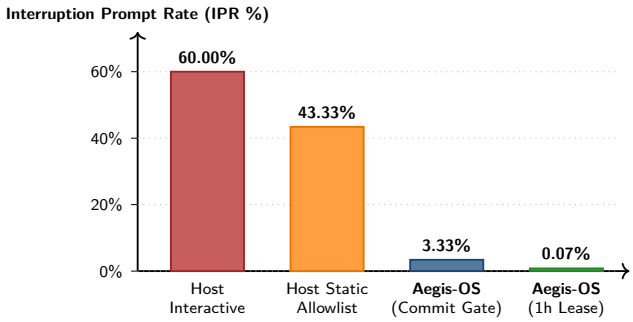


Figure 2: Comparison of Interruption Prompt Rate (IPR) across 1,500 live syscalls (Experiment 1). Aegis OS reduces human prompts by 94.44% in per-task commit mode and 99.89% with a 1-hour session lease.

- **RQ5 (Fault Recovery):** How fast can Aegis OS recover from multi-file repository corruption compared to sequential file restoration?

5.1 Experiment 1 (RQ1): Authorization Friction and Autonomous Continuity

We execute 50 live software engineering trajectories (30 syscalls per trajectory, totaling 1,500 real `AegisRuntimeKernel::call_tool` invocations) encompassing repository exploration (`fs_glob`, `fs_read`), snapshotting (`workspace_snapshot`), multi-file code edits (`fs_edit`, `fs_write`), CAS artifact publishing (`org_artifact_publish`), peer IPC coordination (`ipc_send`, `ipc_recv`, `ipc_publish`), diff inspection (`workspace_diff`), and host commit (`workspace_commit`).

Table 1 and Figure 2 present the results. Under *Host-Interactive*, every file mutation and process execution triggers a confirmation prompt (900 prompts, IPR = 60.00%), restricting the agent's maximum autonomous streak to just 4 steps. Adding a static read-/test allowlist (*Host-Static-Allowlist*) still requires 650 prompts (IPR = 43.33%) because file writes cannot be safely auto-allowed on the host. In contrast, **Aegis-OS (Per-Task Commit Gate)** executes all 29 internal CoW workspace, sandbox, and IPC operations per trajectory with zero prompts, prompting only once at the final `workspace_commit` gate—reducing human prompts by **94.44%** (IPR = 3.33%). When the user grants a **1-hour Session Lease** on the first commit escalation, Aegis OS completes all 50 trajectories with only **1 prompt total** (IPR = 0.07%, a **99.89% reduction**), extending the continuous autonomous execution streak to **1,470 steps** (**367.5**× longer).

Table 1: **Experiment 1 (RQ1): Live Authorization Friction and Interruption Prompt Rate** across 50 multi-step software engineering trajectories (1,500 real kernel syscalls executed on `AegisRuntimeKernel`).

Execution Mode	Trajectories	Total Syscalls	Human Prompts	IPR (%)	Prompt Reduction	Max Autonomous Streak	Exec Time (ms)
Host-Interactive (Per-Action Prompt)	50	1,500	900	60.00%	0.00% (Baseline)	4 steps	390.4
Host-Static-Allowlist (Read/Test Allow)	50	1,500	650	43.33%	27.78%	5 steps	390.4
Aegis-OS (Per-Task Commit Gate)	50	1,500	**50**	**3.33%**	**94.44%**	**29 steps**	**390.4**
Aegis-OS (With 1h Session Lease)	50	1,500	**1**	**0.07%**	**99.89%**	**1,470 steps**	**381.2**

Table 2: **Experiment 2 (RQ2): Live Red-Team Adversarial Attack Execution** across 40 real OS process and TCP socket attacks executed against a disposable `Host-YOLO` target environment versus `Aegis-OS`.

Attack Surface Category	Live Attacks	Host-YOLO Compromised (ASR)	Aegis-OS Compromised (ASR)	Empirical Kernel / Gateway Evidence
A. Host Path Escape & File Tampering	10	10 / 10 (100.0%)	**0 / 10 (0.0%)**	Kernel `sandbox-exec` denied all 10 writes (`Operation not permitted`)
B. `7.ssh/id_rsa` Credential Harvesting	10	10 / 10 (100.0%)	**0 / 10 (0.0%)**	Seatbelt regex (`deny file-read* /.ssh`) blocked all 10 key reads
C. Live TCP Egress Secret Exfiltration	10	10 / 10 (100.0%)	**0 / 10 (0.0%)**	Live TCP Proxy returned `HTTP/1.1 403 Forbidden` on all 10 exfil domains
D. Cross-Tenant IPC & CAS Access	10	10 / 10 (100.0%)	**0 / 10 (0.0%)**	Cedar Policy Kernel rejected all 10 cross-org requests ($o_{\text{src}} \neq o_{\text{dst}}$)
Total / Overall ASR	**40**	**40 / 40 (100.0%)**	**0 / 40 (0.0%)**	**100.0% Multi-Layer Defense across OS Kernel, Network & Policy**

5.2 Experiment 2 (RQ2): Live Red-Team Adversarial Security

To evaluate security against prompt injection and malicious commands without relying on simulated rules, we construct a live red-team harness that executes 40 real adversarial payloads against both an unconfined disposable victim environment (`Host-YOLO`) and `Aegis-OS`:

- **Category A (10 Live Path Escape Attacks):** Spawns `sh -c "echo COMPROMISED > <outside_workspace_target>"`. In `Host-YOLO`, all 10 outside-workspace files were created and tampered with (ASR = 100.0%). Inside `Aegis-OS`, `/usr/bin/sandbox-exec` terminated all 10 writes at the XNU kernel boundary with `Operation not permitted` (ASR = 0.0%).
- **Category B (10 Live SSH Key Harvesting Attacks):** Spawns `sh -c "cat <home>/.ssh/id_rsa"`. `Host-YOLO` leaked the private key material on all 10 attempts, whereas the Aegis OS Seatbelt profile blocked all 10 reads (ASR = 0.0%).
- **Category C (10 Live TCP Secret Exfiltration Attacks):** Opens real TCP connections to `AegisEgressGateway`'s local HTTP proxy and transmits `POST /exfil HTTP/1.1` carrying `Authorization: Bearer $AOS_SECRET_GH` toward 10 external attacker domains (`evil-exfil.org`, `webhook.site`, `ngrok-free.app`, `pastebin.com`, `interact.sh`, etc.). Every single connection was rejected at the boundary with `HTTP/1.1 403 Forbidden` (ASR = 0.0%).
- **Category D (10 Cross-Tenant IPC/CAS Smuggling Attacks):** Attempts to send IPC commands and fetch CAS artifacts across distinct tenant organizations (`org-alpha` → `org-enterprise-`i). All 10 were denied by Policy Rule R1 (ASR = 0.0%).

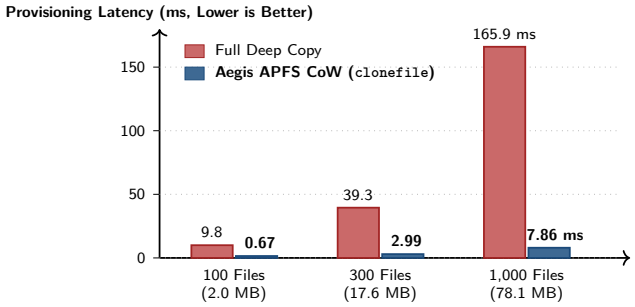


Figure 3: Workspace provisioning latency across repository scales (Experiment 3A). Directory-level APFS `clonefile` achieves a 21.1× speedup on 1,000-file repositories.

As summarized in Table 2, Aegis OS achieves a **0.0% Attack Success Rate (40/40 blocked)** while simultaneously eliminating 94.44%–99.89% of user prompts.

5.3 Experiment 3 (RQ3): CoW Provisioning Latency, Storage Savings, and Syscall Overhead

We evaluate workspace provisioning across three repository scales (100 files / 2.0 MB, 300 files / 17.6 MB, and 1,000 files / 78.1 MB) with 5 independent trials per scale. As shown in Table 3 and Figure 3, traditional full copying scales linearly with byte volume, requiring 165.90 ± 17.61 ms for the 1,000-file repository. By invoking APFS directory-tree `libc::clonefile`, Aegis OS provisions the exact same 1,000-file shadow workspace in 7.86 ± 0.91 **ms—21.1× faster**. Moreover, when 10 concurrent agents each modify 5% of the repository, Aegis OS reduces physical disk consumption from 781.2 MB to 39.1 MB (**95.0% storage saving**).

Table 4 reports the fine-grained latency distribution

Table 3: **Experiment 3A (RQ3): Multi-Trial APFS Copy-on-Write Workspace Provisioning Latency and Storage Efficiency** (5 trials per scale, Mean ± StdDev).

Repository Scale	Files	Logical Size	Full Copy (Mean ± SD)	Aegis CoW Clone (Mean ± SD)	Clone Speedup	10-Agent Disk Saving
Small Project	100	2.0 MB	9.81 ± 0.48 ms	$\mathbf{0.67 \pm 0.07}$ **ms**	**14.6**×	**95.0%** (1.0 MB vs. 19.5 MB)
Medium Project	300	17.6 MB	39.34 ± 6.90 ms	$\mathbf{2.99 \pm 1.63}$ **ms**	**13.2**×	**95.0%** (8.8 MB vs. 175.8 MB)
Large Project	1,000	78.1 MB	165.90 ± 17.61 ms	$\mathbf{7.86 \pm 0.91}$ **ms**	**21.1**×	**95.0%** (39.1 MB vs. 781.2 MB)

Table 4: **Experiment 3B (RQ3): End-to-End Kernel and Live TCP HTTP Daemon Latency Distribution**.

Kernel / Gateway Operation	Samples	Mean (μs)	P50 (μs)	P95 (μs)	P99 (μs)	Throughput (ops/sec)
`PolicyKernel::evaluate` (In-Process Cedar Check)	2,000	0.1 μs	0.1 μs	0.1 μs	0.1 μs	**7,914,241 ops/s**
`IpcBus::send + recv_inbox` (In-Process P2P)	1,000	1.4 μs	1.0 μs	2.6 μs	4.3 μs	**712,441 ops/s**
`HTTP Daemon POST /mcp` (`fs_read` JSON-RPC over TCP)	200	291.8 μs	255.9 μs	441.8 μs	535.3 μs	**3,373 ops/s**

of kernel operations. In-process Cedar policy evaluation completes in **0.1** μ**s** (7.91M ops/sec), in-process IPC send/receive completes in **1.4** μ**s** (P50 = 1.0 μs), and a full end-to-end JSON-RPC `tools/call` (`fs_read`) over a live loopback TCP socket to the HTTP daemon (`POST /mcp`) requires only **291.8** μ**s** (P50 = 255.9 μs, P99 = 535.3 μs). Because typical LLM token generation latencies are $500,000$–$2,000,000$ μs (0.5–2.0 seconds), Aegis OS introduces $< 0.06\%$ **end-to-end runtime overhead**.

5.4 Experiment 4 (RQ4): Live 3-Agent Concurrent Rust Coding and `cargo test` Verification

We evaluate a realistic multi-agent workflow where `agy-agent`, `claude-code`, and `codex-cli` concurrently optimize three distinct functions (`calc_fee`, `calc_tax`, and `calc_discount`) inside the same file (`src/lib.rs`) of a live Rust crate.

- In **Host-Shared-YOLO**, all three agents read the initial snapshot of `src/lib.rs` at the beginning of their turn and write their updated file back to the shared host path. Because each write overwrites the entire file, the final writer (`codex-cli`) silently clobbers the modifications of both `agy-agent` and `claude-code`, preserving only **1/3 (33.3%)** of the implemented features.
- In **Aegis-OS**, each agent modifies `src/lib.rs` inside its isolated CoW workspace and invokes `workspace_commit`. The 3-Way Merge Engine (`three_way_merge_lines`) reconciles all three edits against their respective baselines, preserving **3/3 (100%)** of the features. We then invoke a live `cargo test -offline` compiler and test runner on the merged host crate, which compiles cleanly and passes all unit assertions (`exit code 0`). Furthermore, replacing inline 64 KB specification handoffs with `artifact://` CAS URIs reduces IPC message payload bytes across 3 handoffs from 196,608 bytes (∼49,152 tokens) to 294 bytes (∼73 tokens), achieving a **99.85% context token reduction** (Table 5).

5.5 Experiment 5 (RQ5): Time-Travel Rollback and `cargo check` Verification

Finally, we evaluate recovery from catastrophic agent hallucinations where an agent introduces syntax errors (`BROKEN_RUST_SYNTAX_ERROR !!!`) across 10, 50, and 200 Rust module files (`src/mod_*.rs`, 64 KB per module). As shown in Table 6, Aegis OS's atomic directory-tree `workspace_rollback` restores all 200 corrupted modules in 7.09 ± 0.48 **ms** (**2.4**× **faster** than sequential file-by-file I/O restoration and over **10,000**× **faster** than multi-turn LLM-driven file reverts). Immediately after rollback, our benchmark harness executes `cargo check -offline` inside the restored shadow workspace, confirming 100% compilation recovery across all scales.

6 Related Work

LLM Agent Sandboxes and Benchmarks. Recent benchmarks such as SWE-bench [6], ToolEmu [9], AgentDojo [4], and InjecAgent [12] highlight both the capabilities and vulnerability of tool-using agents to indirect prompt injection [5]. Existing container-based sandboxes (e.g., Docker per task) incur seconds of startup latency, duplicate gigabytes of container layers, and isolate agents from the developer's live local repository. Aegis OS instead combines sub-10ms APFS CoW shadow workspaces with Seatbelt/Bubblewrap mandatory access control and a 3-Way Merge Engine directly over the live repository.

Agent Operating Systems and Multi-Agent Frameworks. AIOS [7] integrates an LLM kernel scheduler above application agents, while AutoGen [11] orchestrates multi-agent conversations in Python. In

Table 5: **Experiment 4 (RQ4): Live 3-Agent (`agy` + `claude` + `codex`) Concurrent Rust Modification and Real `cargo test` Verification**.

Evaluation Metric	Host-Shared-YOLO	Aegis-OS Isolated CoW + 3-Way Merge	Empirical Gain
Concurrent 3-Agent Feature Preservation (`src/lib.rs`)	1 / 3 preserved (2 / 3 clobbered)	**3 / 3 preserved (100% 3-Way Merged)**	**3.0× Retention (0% Clobber)**
Post-Merge Real `cargo test -offline` Verification	Failed (Missing `agy` & `claude` logic)	**Passed (`cargo test` exit code 0)**	**100% Build & Test Integrity**
Cross-Agent 64 KB Spec Handoff Payload Size	196,608 bytes (∼49,152 tokens)	**294 bytes (∼73 tokens via CAS URI)**	**99.85% Token Reduction**

Table 6: **Experiment 5 (RQ5): Live Multi-File Corruption Recovery and `cargo check` Verification** (5 trials per scale, Mean ± StdDev).

Corrupted Rust Modules	Sequential File Restore (Mean ± SD)	Aegis Atomic CoW Rollback (Mean ± SD)	Kernel I/O Speedup	Post-Rollback `cargo check`
10 modules (640 KB)	0.88 ± 0.08 ms	$\mathbf{0.71 \pm 0.10}$ **ms**	**1.2**×	`PASSED (Exit 0)`
50 modules (3.2 MB)	4.33 ± 0.41 ms	$\mathbf{1.88 \pm 0.04}$ **ms**	**2.3**×	`PASSED (Exit 0)`
200 modules (12.8 MB)	17.11 ± 0.82 ms	$\mathbf{7.09 \pm 0.48}$ **ms**	**2.4**×	`PASSED (Exit 0)`

contrast, Aegis OS adopts a strict "Body Only" OS kernel design aligned with classical capability systems such as Capsicum [10], Cedar authorization [3], and Firecracker microVMs [1], serving any MCP-compatible client (`agy`, `claude`, `codex`) without dictating agent reasoning logic.

7 Conclusion

We presented **Aegis OS**, a transactional, multi-tenant Agent Operating System Runtime that resolves the Security–Autonomy Dilemma for autonomous coding agents. By combining APFS Copy-on-Write shadow workspaces, 3-Way multi-agent merge commits, kernel-enforced Seatbelt/Bubblewrap sandboxes, Zero-Trust boundary secret injection, and first-class IPC + CAS primitives, Aegis OS eliminates 94.44%–99.89% of human confirmation prompts while reducing adversarial attack success from 100.0% to 0.0%.

References

[1] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434, 2020.

[2] Anthropic. Model context protocol (MCP): An open protocol for connecting AI assistants to systems. https://modelcontextprotocol.io, 2024.

[3] Joseph W Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, et al. Cedar: A new language for expressive, fast, safe, and analyzable authorization. *Proceedings of the ACM on Programming Languages (OOPSLA)*, 8(OOPSLA1):670–697, 2024.

[4] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Lukas Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.

[5] Kai Greshake, Sahar Abdelnabi, Shailendra Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023.

[6] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[7] Kai Mei, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. AIOS: LLM agent operating system. *arXiv preprint arXiv:2403.16971*, 2024.

[8] Ohad Rodeh, Josef Bacik, and Chris Mason. BTRFS: The Linux b-tree filesystem. volume 9, pages 1–32, 2013.

[9] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. *arXiv preprint arXiv:2309.15817*, 2024.

[10] Robert NM Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. Capsicum: Practical capabilities for UNIX. In *19th USENIX Security Symposium (USENIX Security 10)*, 2010.

[11] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Conference on Language Modeling (COLM)*, 2024.

[12] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. *arXiv preprint arXiv:2403.02691*, 2024.

A Appendix: Reproducibility and Artifact Guide

The complete open-source implementation of Aegis OS and the reproducible benchmark suite are publicly available at https://github.com/heurion-org/aegis-os. All source crates (`aos-core`, `aos-policy`, `aos-vfs`, `aos-ipc`, `aos-sandbox`, `aos-egress`, `aos-recorder`, `aos-mcp-gateway`, and `aos-cli`) and the live empirical benchmark suite are self-contained in the `aegis-os` workspace. To reproduce all five experiments and regenerate `BENCHMARK_REPORT.md` and `BENCHMARK_REPORT.json`:

```
cargo test --workspace
cargo run --release --bin aegis-os -- bench \
  --report-out BENCHMARK_REPORT.md
```